

CS 97SI: Introduction to Programming Contests

Course Information and Announcements

- Instructor: [Jaehyun Park](#) (Stanford ACM-ICPC coach)
- Subscribe to the [Stanford ACM-ICPC email list](#) to get notifications about future practice contests.
- (Added on 8/21/2013) This class was taught in 2011-12 Winter. I'm getting a lot of emails asking if I'm teaching it again, but there is no plan to offer the course at the moment.
- (Added on 6/30/2015) All the slides are rewritten in LaTeX now.

Lecture slides

1. [Introduction](#)
2. [Mathematics](#)
3. [Data Structures](#)
4. [Dynamic Programming \(DP\)](#)
5. [Combinatorial Games](#)
6. [Basic Graph Algorithms](#)
7. [Shortest Path Algorithms](#)
8. [Network Flow Problems](#)
9. [Computational Geometry](#)
10. [String Algorithms](#) (Additional material: [Suffix Arrays - A Programming Contest Approach](#))

Practice Problems

All the problems below are from [Peking Online Judge \(POJ\)](#). Submissions should be made directly to the automated judging system. Problems are classified into 10 different categories, and the lectures will cover essential algorithms and theoretical background for each particular category. The numbers in parentheses represent the difficulty of the problems (0: easiest, 10: hardest). The difficulty rating is subjective; you might find a 5-rated problem easier than a 4-rated one. But do not attempt the challenge problems unless you are sure that you can solve *all* the easier ones. If POJ is down, you can use the offline copies of the problems [here](#).

- [Coding Exercise](#)
- [Mathematics](#)
- [Data Structures](#)
- [Dynamic Programming \(DP\)](#)
- [Combinatorial Games](#)
- [Basic Graph Algorithms](#)
- [Shortest Path Algorithms](#)
- [Network Flow Problems](#)
- [Computational Geometry](#)
- [String Algorithms](#)

Common coding mistakes

- Not initializing variables
- Using 32-bit integers instead of 64-bit ones
- Using out-of-bound array indices
- Using a semicolon after a for loop

```
for(i = 0; i < n; i++);  
    some code
```

- Reusing the same variable in nested for loops

```
for(i = 0; i < 1000; i++)  
    for(i = 0; i < 10; i++)  
        some code
```

- Not using `break` in a switch-case statement (just don't use switch-case statements for programming contests)
- Not taking (very) small cases into account (e.g. $n = 0, 1$)
- Writing `arr[j][i]` instead of `arr[i][j]`, and similar errors
- Writing a given constant incorrectly (e.g. taking modulo 12345789 instead of 123456789)
- Bad parentheses in macro definition

```
#define min(a, b) a<b?a:b // incorrect  
#define min(a, b) (((a)<(b))?(a):(b)) // correct
```

- Writing `cos(180)` instead of `cos(pi)`

If you get stuck...

- First, remember that you are **not** stuck unless you have spent more than a day on a single problem. It is perfectly normal to spend many hours on just coming up with the right algorithm.
- Read the problem statement again. You might have missed something very important.
- Carefully go through the lecture slides. Think about how to apply the basic algorithms covered in class to the problem.
- Discuss the problem with friends. Working in teams of size 2 or 3 is strongly recommended. As long as you don't copy-paste someone else's codes blindly, you can even share your (possibly buggy) codes.

Links

- [Stanford Local Programming Contest](#)
- [Stanford ACM Team Notebook](#)
- [Peking Online Judge](#)
- [USACO Training Gate](#)

